

Computability - Recitation 2

April 20, 2026

1 Definitions: Nondeterministic Finite Automaton (NFA)

Recall we defined in class an NFA as a 5-tuple $\mathcal{A} = \langle Q, \Sigma, \delta, Q_0, F \rangle$, such that

- Q is a set of states,
- Σ is an alphabet,
- $Q_0 \subseteq Q$ is a set of initial states,
- $\delta : Q \times \Sigma \rightarrow 2^Q$ is a transition function,
- and $F \subseteq Q$ is a set of accepting states.

Let $w = w_1 \dots w_n \in \Sigma^*$ be an input word. A *run* of $\mathcal{A}$ on w is a sequence of states $r_0, \dots, r_n$ such that:

- Initial state: $r_0 \in Q_0$.
- Transition: $r_{i+1} \in \delta(r_i, w_{i+1})$ for every $0 \leq i < n$.

A run is called *accepting* if $r_n \in F$.

An NFA can have multiple different runs on an input w . We say that $\mathcal{A}$ *accepts* w if there exists at least one accepting run.

2 Example: a small NFA can replace a large DFA

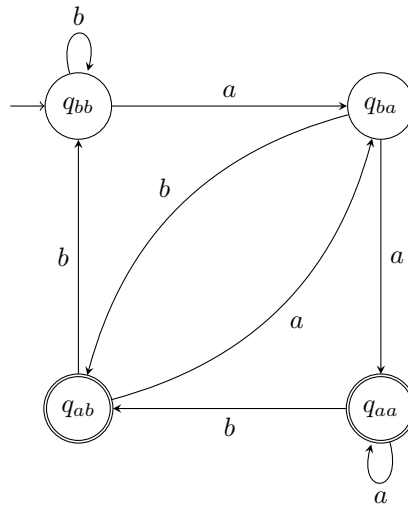
Exercise

For every $k \in \mathbb{N}$, we define L_k to be the language over $\Sigma = \{a, b\}$ that contains the words whose k th letter from the end is a .

1. Design a DFA that decides L_2 .
2. Design an NFA that decides L_2 .
3. For a general k , show that there exists an NFA with $k + 1$ states that decides L_k .
4. For a general k , show that any DFA that decides L_k has at least 2^k states.

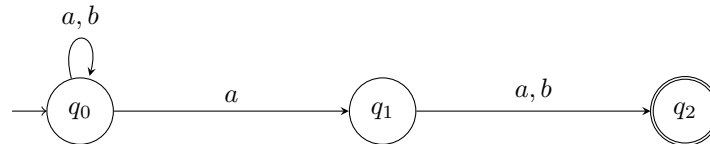
Solution

1. The following DFA decides L_2 .



It keeps track of the last two letters of the word to determine whether the second letter from the end is a . Each state contains the words that end with some pair of letters (treating words with less than two letters as they begin with a sequence of bs). For example, the state q_{ba} contains the word a and the words that end with ba . Make sure you understand why this is the case (and that you can prove it by induction).

2. The following NFA decides L_2 .

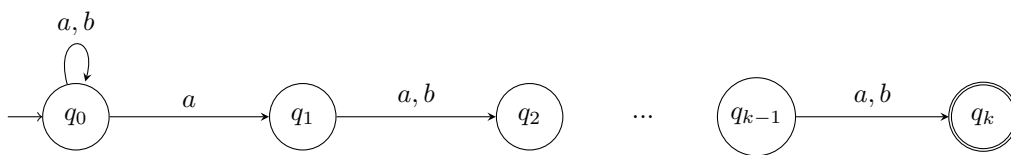


We explain (notice: this is not a full formal proof). There exists an accepting run of the NFA for a word w if and only if its second letter from the end is a . To see that every such word has an accepting run, we can show

that $\overbrace{q_0, \dots, q_0}^{|w|-1 \text{ times}}, q_1, q_2$ is a valid and accepting run. To see that only such words have an accepting run, we can show that every accepting run ends with q_0, q_1, q_2 , and hence the word ends either with aa or ab .

A useful intuition is that the NFA uses its non-determinism to "guess" when to move to q_1 , that is, to guess which letter is the second letter from the end of the word.

3. The following NFA with $k + 1$ states that decides L_k :



The idea is the same as in section 2.

To formally prove the correctness of the NFA, we use induction on the length of the word, as we did for DFAs. For DFAs, for each state, we characterized the set of words that end their run in this state. For NFAs, for each subset of states, we characterize the words that *may* end their run in each of the states in the set.

For every word $w \in \Sigma^*$, we denote the positions from the end where a appear in the last k letters of w by $i_1, \dots, i_l$. For example, the positions in $w = baba$ are 1, 3 because a is the last letter and the third letter from the end. We claim that words whose a positions from the end are $i_1, \dots, i_l$ can reach $\{q_0, q_{i_1}, \dots, q_{i_l}\}$.

For the induction step, we decompose the word w as $w = u \cdot \sigma$ where $\sigma \in \{a, b\}$, and identify the positions of a in u from the end. By the induction hypothesis, we determine the possible sets of states that u can reach. Then, for each $\sigma \in \{a, b\}$, we verify that the states that δ may lead w to are exactly those in the induction claim.

The full proof is left as an exercise.

4. Assume by contradiction that there exists a DFA $\mathcal{A}$ that decides L_k with less than 2^k states. Notice that there are 2^k words over $\{a, b\}^*$ from length k . Using the pigeonhole principle, there are two different words from length k , $u = u_0 \dots u_k \neq v = v_0 \dots v_k \in \{a, b\}^*$ such that $\mathcal{A}$ end its run on them in the same state. Denote

by i the first index in which the words are different, without loss of generality $u_i = a$ and $v_i = b$. If we add the same suffix to both words, the resulting words still end in the same state, and hence are accepted or rejected together. In particular, $\mathcal{A}$ accepts or rejects the words $u' := u \circ a^{i-1}$ and $v' := v \circ a^{i-1}$ together. However, the k th letter from the end of u' is a so $\mathcal{A}$ accepts it. Hence, $\mathcal{A}$ should also accept v' but its k th letter from the end is b so it is not in L_k in contradiction to our assumption. Therefore, every DFA that decides L_k has at least 2^k states.

This example highlights the advantage of non-determinism. It enables us to create much more compact and simple models, which can be simpler to understand and to prove claims for.

3 Closure properties of NREG

3.1 Union

You have seen in class that REG is closed under union, using a product automaton. We now show that NREG is closed under union, as well (without using the fact that REG = NREG, which we haven't shown yet).

Let $L_1, L_2 \subseteq \Sigma^*$, then

$$L_1 \cup L_2 = \{w : w \in L_1 \text{ or } w \in L_2\}$$

We want to show that if $L_1, L_2 \in \text{NREG}$, then there exists an NFA that recognizes $L_1 \cup L_2$. Let $\mathcal{A} = \langle Q, \Sigma, \delta, Q_0, F \rangle$ and $\mathcal{B} = \langle P, \Sigma, \eta, P_0, G \rangle$ be NFAs such that $L(\mathcal{A}) = L_1$ and $L(\mathcal{B}) = L_2$. We can assume without loss of generality that $Q \cap P = \emptyset$ (otherwise we can just rename states).

The idea is to define a new NFA which is the “union” of both NFAs. Formally,

$$\mathcal{C} = \langle Q \cup P, \Sigma, \alpha, Q_0 \cup P_0, F \cup G \rangle$$

where for every $q \in Q \cup P, \sigma \in \Sigma$,

$$\alpha(q, \sigma) = \begin{cases} \delta(q, \sigma) & q \in Q \\ \eta(q, \sigma) & q \in P \end{cases}$$

Let $w = w_1 \dots w_n \in \Sigma^*$ and let $r_0, \dots, r_n$ be a run of $\mathcal{C}$ on w . We can show inductively that the entire run is contained in Q or contained in P : for all $0 \leq i < n$, if $r_i \in Q$ then $r_{i+1} \in \alpha(r_i, w_{i+1}) = \delta(r_i, w_{i+1}) \subseteq Q$. Hence, if $r_0 \in Q$ then $r_0, \dots, r_n \in Q$. Similarly for P .

We show that $L_1 \cup L_2 = L(\mathcal{C})$ by two-sided containment.

- Let $w = w_1 \dots w_n \in L_1 \cup L_2$. Without loss of generality, suppose $w \in L_1$, so there is an accepting run $r_0, \dots, r_n$ of $\mathcal{A}$ on w . We claim that this run is also an accepting run of $\mathcal{C}$ on w . Indeed, the initial state is $r_0 \in Q_0 \subseteq Q_0 \cup P_0$, and each transition holds:

$$r_{i+1} \in \delta(r_i, w_{i+1}) = \alpha(r_i, w_{i+1})$$

Since the run is accepting, the last state r_n is in F , so it is also in $F \cup G$. Thus $w \in L(\mathcal{C})$.

- Let $w = w_1 \dots w_n \in L(\mathcal{C})$, and let $r_0, \dots, r_n$ be an accepting run of $\mathcal{C}$ on w . Then $r_n \in F \cup G$. Assume without loss of generality that $r_n \in F$. The entire run is contained in Q or in P , and since $r_n \in F \subseteq Q$, we have $r_i \in Q$ for all $0 \leq i \leq n$. We claim that $r_0, \dots, r_n$ is also an accepting run of $\mathcal{A}$ on w . Indeed, $r_0 \in Q_0 \cup P_0$, and since the entire run is in Q then $r_0 \in Q_0$. For every $0 \leq i < n$, we have $r_{i+1} \in \alpha(r_i, w_{i+1})$. Since $r_i \in Q$, we have $r_{i+1} \in \delta(r_i, w_{i+1})$. Thus $w \in L(\mathcal{A})$, and we are done.

3.2 Concatenation

Let $L_1, L_2 \subseteq \Sigma^*$, and let $\mathcal{A} = \langle Q, \Sigma, \delta, Q_0, F \rangle$ and $\mathcal{B} = \langle P, \Sigma, \eta, P_0, G \rangle$ be NFAs that recognize L_1, L_2 respectively. We want to construct an NFA $\mathcal{C}$ such that $L(\mathcal{C}) = L(\mathcal{A}) \cdot L(\mathcal{B})$.

Intuitively, we want to “run $\mathcal{A}$ and then $\mathcal{B}$ ”. The issue is that there are multiple ways to split a word into two parts, thus it's unclear when to move from $\mathcal{A}$ to $\mathcal{B}$. We use nondeterminism to “guess” when to switch from an accepting state of $\mathcal{A}$ to an initial state of $\mathcal{B}$.

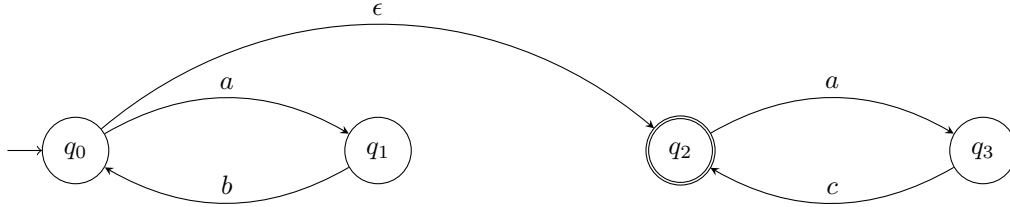
$\mathcal{C}$ is defined over Σ , its states are the union $Q \cup P$. We take the initial states Q_0 from $\mathcal{A}$, and the accepting states G from $\mathcal{B}$. The transitions of $\mathcal{C}$ include the transitions of δ and η , along with ϵ -transitions from each accepting state of $\mathcal{A}$ to each initial state of $\mathcal{B}$.

The proof of correctness is left as an exercise.

For example, consider $L_1 = (ab)^*$ and $L_2 = (ac)^*$ over $\Sigma = \{a, b, c\}$, and the following automata:



Let's apply the construction described above for this concrete example:

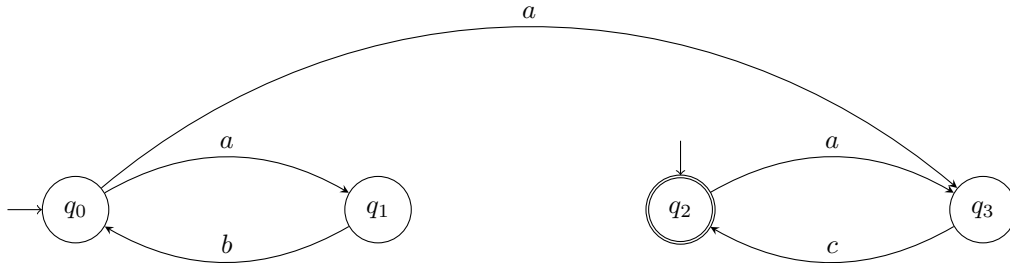


We used ϵ -transitions to describe the automata above, in words and when drawing it, but our formal definition did not allow such transitions. How can we settle this problem? In the next section, we will see that any NFA with ϵ -transitions can be transformed into an NFA without ϵ -transitions which can be described formally.

4 Formalizing ϵ -transitions

Let's start with the example we just saw. How can we describe an equivalent NFA without ϵ -transitions? The key insights are:

- the ϵ -transition from q_0 to q_2 means that any transition that leads to q_0 "could also" lead to q_2 . Therefore, since there is a b -transition from q_1 to q_0 , we add a b -transition from q_1 to q_2 .
- Since q_0 is an initial state, and q_2 is reachable from it using an ϵ -transition, it should also be an initial state.



Let's generalize and formalize these idea, but first we recall that an NFA is really just the 5-tuple containing the alphabet, states, etc. The graph with the arrows is just a way to represent this 5-tuple, and we can decide how we chose to interpret the graph as a an NFA. We can chose any way which makes sense, so the following is just one option which we like and shall use in this course. Given a description of an NFA as a label arrow grfaph with ϵ -transitions, we denote the non- ϵ transitions of it by $\eta : Q \times \Sigma \rightarrow 2^Q$. We define the actual NFA that the graph represents as $\langle Q, \Sigma, \delta, Q_0, F \rangle$, where the states Q , the alphabet Σ , and the accepting states F remain as they are. The set of starting states Q_0 is define to contain all the states that are marked as starting states in the graph, together with all the states reachable from them by a path that only contains ϵ marked arrows.

As for the transition rule δ , we define the set $\delta(q, \alpha)$ to contain all the states reachable from q in the graph using an number of ϵ labeled arrows, and exactly one α -labeled arrow. We can write this in another way which some may prefer: let

$$E(q) = \{q' \in Q : q' \text{ is reachable from } q \text{ using only } \epsilon\text{-transitions}\}$$

, and let η be the transition function that the graph would define if we just erase all ϵ transitions. Then

$$\delta(q, \sigma) = \bigcup_{q' \in E(q)} E(\eta(q', \sigma))$$

Using the same notation, if P_0 is the set of states marked as initial states in the graph, then

$$Q_0 = \bigcup_{q \in P_0} E(q)$$

5 The function δ^* for NFAs

In the previous recitation, we defined δ^* for DFAs. We now do the same for NFAs. This will help us understand better the runs of NFAs, as preparation for determinization (constructing an equivalent DFA). Let $\mathcal{A} = \langle Q, \Sigma, Q_0, \delta, F \rangle$ be an NFA. The function $\delta^* : 2^Q \times \Sigma^* \rightarrow 2^Q$ takes a set of states and a word, and gives the set of states we can reach using that word. Let $S \subseteq Q$ and $w = w_1 \dots w_n \in \Sigma^*$. In class, we defined:

$$\delta^*(S, w) = \{q \in Q : \text{there is a run } r_0, \dots, r_n \text{ such that } r_0 \in S, r_n = q, \text{ and } r_{i+1} \in \delta(r_i, w_{i+1}) \text{ for all } 0 \leq i < n\}$$

Another way to define δ^* is recursively:

$$\delta^*(S, w) = \begin{cases} S & w = \epsilon \\ \bigcup_{q \in \delta^*(S, w')} \delta(q, \sigma) & w = w' \cdot \sigma \end{cases}$$

Proposition 1. *The above definitions are equivalent.*

Proof. We show that the first definition satisfies both the base case and the recursion written in the second definition. Since they agree on both, they have the same function. In the following, δ^* refers to the first definition, and $S \subseteq Q$.

For the base case, let $w = \epsilon$. A run on ϵ starts at some state $r_0 \in S$ and immediately stops. Therefore the reachable states are exactly the states in S , and we have $\delta^*(S, \epsilon) = S$.

For the recursive case, let $w = w' \cdot \sigma$. We want to prove that:

$$\delta^*(S, w'\sigma) = \bigcup_{q \in \delta^*(S, w')} \delta(q, \sigma)$$

Two-sided containment:

- Let $r_n \in \delta^*(S, w'\sigma)$. There is a run $r_0, \dots, r_n$ on $w' \cdot \sigma$ that starts in S , where $r_n \in \delta(r_{n-1}, \sigma)$. Notice that $r_0, \dots, r_{n-1}$ is a run on w' starting in S , so $r_{n-1} \in \delta^*(S, w')$. Therefore,

$$r_n \in \delta(r_{n-1}, \sigma) \subseteq \bigcup_{q \in \delta^*(S, w')} \delta(q, \sigma)$$

- Let $r_n \in \bigcup_{q \in \delta^*(S, w')} \delta(q, \sigma)$. Then there is some $r_{n-1} \in \delta^*(S, w')$ for which $r_n \in \delta(r_{n-1}, \sigma)$. There is a run $r_0, \dots, r_{n-1}$ from S on w' , and since $r_n \in \delta(r_{n-1}, \sigma)$, we have that $r_0, \dots, r_n$ is a run from S to r_n . Therefore $r_n \in \delta^*(S, w' \cdot \sigma)$.

□

Proposition 2. *Let $\mathcal{A} = \langle Q, \Sigma, Q_0, \delta, F \rangle$ be NFA. Then the language of the NFA is:*

$$L(\mathcal{A}) = \{w \in \Sigma^* : \delta^*(Q_0, w) \cap F \neq \emptyset\}$$

Proof. $\delta^*(Q_0, w)$ contains exactly the states reachable from Q_0 in a run on w . By definition, $\mathcal{A}$ accepts w if and only if there exists a state such that is also in F .

□

6 Extra Example (if time permits)

Given a regular language L , show that

$$L_{\text{mix}} := \{w \in \Sigma^* : \exists z \in \Sigma^{|w|} \text{ s.t. } z_1 w_1 \dots z_{|w|} w_{|w|} \in L\} \in \text{NREG}.$$

6.1 Solution

Let $\mathcal{A} = \langle Q, \Sigma, \delta, q_0, F \rangle$ be a DFA that recognizes L . Define an NFA $\mathcal{B} = \langle Q, \Sigma, \eta, \{q_0\}, F \rangle$ where:

$$\eta(q, \sigma) = \{\delta(\delta(q, \alpha), \sigma) : \alpha \in \Sigma\}$$

The idea is that when reading a letter $\sigma = w_i$, the NFA first “guesses” a letter $\alpha = z_i$ and goes to a state that the DFA would reach after reading $z_i w_i$.

We prove that $L(\mathcal{B}) = L_{\text{mix}}$ by containments:

- Let $w = w_1 \cdots w_n \in L_{\text{mix}}$. Then there exists $z = z_1 \cdots z_n$ such that $z_1 w_1 \cdots z_n w_n \in L$. Let $r_0, s_1, r_1, s_2, r_2, \dots, s_n, r_n$ be an accepting run of $\mathcal{A}$ on $z_1 w_1 \cdots z_n w_n$. We claim that $r_0, r_1, \dots, r_n$ is an accepting run of $\mathcal{B}$ on w . Indeed, $r_0 \in \{q_0\}$ and $r_n \in F$, and for every $0 \leq i < n$ we have:

$$r_{i+1} = \delta(s_{i+1}, w_{i+1}) = \delta(\delta(r_i, z_{i+1}), w_{i+1})$$

Therefore $r_{i+1} \in \eta(r_i, w_{i+1})$ by definition of η (choosing $\alpha = z_{i+1}$). So it is a valid accepting run and we have $w \in L(\mathcal{B})$.

- Let $w = w_1 \cdots w_n \in L(\mathcal{B})$. Let $r_0, \dots, r_n$ be an accepting run of $\mathcal{B}$ on w . For every $0 \leq i < n$, we have $r_{i+1} \in \eta(r_i, w_{i+1})$. Hence there exists a letter $z_{i+1} \in \Sigma$ such that $r_{i+1} = \delta(\delta(r_i, z_{i+1}), w_{i+1})$. Denote $s_{i+1} = \delta(r_i, z_{i+1})$.

We claim that $r_0, s_1, r_1, s_2, r_2, \dots, s_n, r_n$ is an accepting run of $\mathcal{A}$ on $z_1 w_1 \cdots z_n w_n$ (hence $w \in L_{\text{mix}}$). Indeed, we have $r_0 \in \{q_0\}$ so r_0 is the initial state, and $r_n \in F$ is an accepting state. For every $0 \leq i < n$:

$$\begin{aligned} r_{i+1} &= \delta(\delta(r_i, z_{i+1}), w_{i+1}) = \delta(s_{i+1}, w_{i+1}) \\ s_{i+1} &= \delta(r_i, z_{i+1}) \end{aligned}$$

So it is a valid accepting run. This choice of z shows that $w \in L_{\text{mix}}$.

Hence $L_{\text{mix}} \in \text{NREG}$.